



Aprendizaje Automático 1

Práctica 3 - Clasificación Supervisada

Comienzo: 30 noviembre (grupo jueves), 1 diciembre (grupo viernes)

Entrega: a través de Moodle. 26 noviembre (grupo jueves), 27 noviembre (grupo viernes)

Semana 1

El objetivo en esta práctica es desarrollar un modelo que estima la probabilidad de que un cliente abandone una empresa telefónica y se vaya a la competencia (problema de fuga de clientes o “churn”). Para ello nos proporcionan una base de datos, **fuga_clientes_empresa_telefonica_construccion.csv**, con datos históricos etiquetados.

Los campos de esta base de datos son:

- Customer ID
- network_age
- Customer tenure in month
- Total Spend in Months 1 and 2 of 2017
- Total SMS Spend
- Total Data Spend
- Total Data Consumption
- Total Unique Calls
- Total Onnet spend
- Total Offnet spend
- Total Call centre complaint calls
- Network type subscription in Month 1
- Network type subscription in Month 2
- Most Loved Competitor network in in Month 1
- Most Loved Competitor network in in Month 2
- **Churn Status**

La variable a predecir es **Churn Status**: el valor es 1 si el cliente **abandona** la compañía, 0 si no la abandona.

Por otra parte, se nos proporciona otra base de datos,

fuga_clientes_empresa_telefonica_explotacion.csv, con información sobre clientes de los que no se sabe ahora mismo si van a permanecer o no en la compañía. Por tanto en esta segunda base de datos todos los valores de la columna **Churn Status** son missing (NaN).

La compañía nos pide que proporcionemos los IDs de los 100 clientes de la base de datos de explotación que tengan más probabilidad de abandonar la compañía. Para ello proporcionaremos como entregable a la compañía un archivo csv con una sola columna, **Customer ID**, y 100 filas que contengan los IDs de los clientes seleccionados.

La calidad de las predicciones la medirá la compañía dentro de 1 mes, chequeando cuáles de esos 100 clientes se van realmente de la empresa.



En la semana 1 seguiremos los siguientes pasos:

Paso 1: Limpieza del dataset

Se aplicarán los métodos aprendidos en las prácticas anteriores para eliminar variables colineales o altamente redundantes, gestionar outliers, recodificar variables, etc. Importante: todo el preprocesamiento que se realice con el dataset de construcción habrá que realizarlo posteriormente con el de explotación.

Paso 2: Construcción y análisis de un modelo dummy usando la librería scikit-learn (sklearn)

- Construye con el modelo DummyClassifier (sklearn.dummy.DummyClassifier) un modelo que predice siempre la clase más frecuente
- Calcula sus scores de clasificación en el dataset de entrenamiento y de test. ¿Hay sobreajuste?
- Analiza las predicciones de clase que da en el conjunto de test (model.predict) y las estimaciones de las probabilidades de las clases (model.predict_proba). Encuentra qué relación hay entre esas salidas y las estadísticas de entrenamiento.

Paso 3: Construcción y análisis de un modelo de regresión logística usando la librería scikit-learn (sklearn)

- Construye con sklearn un modelo de regresión logística (clase sklearn.linear_model.LogisticRegression). No te olvides de estandarizar las variables de entrada. ¿Cuáles son los scores obtenidos en entrenamiento y en test? ¿Hay sobreajuste? ¿Cómo se comparan con los obtenidos con el DummyClassifier?
- Si no se especifica en LogisticRegression el parámetro C, este es 1, indicando un grado medio de regularización. Al igual que con la regresión lineal, podemos hacer que sklearn ajuste automáticamente ese parámetro de regularización usando LogisticRegressionCV. ¿Qué obtienes ahora? ¿es mejor que el anterior modelo?

Semana 2

En esta semana realizaremos un análisis más detallado del modelo de clasificación.

Paso 1. Construcción y análisis de un modelo de regresión logística usando la librería statsmodels

- Construye un modelo de regresión logística usando la librería statsmodels (clase Logit de statsmodels.api). ¿Obtiene los mismos coeficientes que sklearn? ¿Los scores son los mismos?
- Saca por pantalla la información estadística del modelo: desviación estándar de los coeficientes, valores p, intervalos de confianza de los coeficientes. Con una



significancia estadística del 95%, ¿qué variables parece que no aportan información?

Paso 2: Selección de variables con librería statsmodels

- Implementa un algoritmo similar al que implementaste en la práctica 2 para la selección de variables con statsmodels:
 1. Construye modelo con variables cols
 2. Detecta qué variables parece que no aportan información de acuerdo al criterio de los valores P. Si no encuentras ninguna, fin del algoritmo. Si encuentras una o varias, toma la que tenga un valor P mayor y quítala de cols. Vuelve a 1
- ¿Con cuántas variables te quedas usando este algoritmo? ¿Qué scores obtienes? ¿Es mejor que el modelo sin seleccionar variables?

Paso 3

Usando el mejor modelo de regresión logística que hayas obtenido hasta ahora, extrae las siguientes métricas compáralas con el modelo dummy:

- Matriz de confusión
 - https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html
 - Usaremos solo los dos primeros parámetros y_true, y_pred
- Curva ROC y área bajo la curva (AUC)
 - https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_curve.html
 - Usaremos solo los parámetros y_true, y_score (las probabilidades del modelo para la clase "positiva"), y pos_label (cuál de las dos clases es la "positiva").
 - Una vez calculado el TPR y el FPR habría que representar la gráfica de TPR (eje vertical) en función del FPR (eje horizontal).
 - Para calcular el AUC (área bajo la curva):
 - https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html
 - Usaremos solo los parámetros y_true e y_score
- F1
 - https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html
 - Usaremos los parámetros y_true, y_pred
- Curva de precision-recall
 - https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_recall_curve.html
 - Usaremos los parámetros y_true, probas_pred (probabilidades predichas para la clase positiva), pos_label (cuál de las dos clases es la "positiva").



- Una vez calculado el precision y el recall habría que representar la gráfica de Precision (eje vertical) en función del Recall (eje horizontal).

Paso 4.

Implementa a mano (sin ayuda de funciones de sklearn) el cálculo de la curva ROC y su AUC.

Ayuda (ejemplo de cálculo de curva ROC):

Clase real y probabilidad de clase=1 predicha por el modelo. Asumimos que la clase positiva (clase a detectar) es la 1:

Los posibles umbrales de alarma para el modelo son 0.1, 0.2, 0.35, 0.4, 0.8. Incluiremos también el umbral infinito para calcular el caso en el que no hay ninguna alarma. Tenemos ahora:

Clase real	Pmodelo(clase=1)
0	0.1
0	0.2
0	0.4
1	0.35
1	0.8

Umbral	# Alarmas	TP	TPR	FP	FPR
Infinito	0	0	0	0	0
0.8	1	1	1/2	0	0/3
0.4	2	1	1/2	1	1/3



Umbral	# Alarmas	TP	TPR	FP	FPR
0.35	3	2	2/2	1	1/3
0.2	4	2	2/2	2	2/3
0.1	5	2	2/2	3	3/3

¿Coinciden la curva ROC y el AUC implementados a mano con lo que obtienes con sklearn?

Paso 5.

Suponiendo que un falso positivo (un cliente que se predice que va a irse de la empresa pero se queda) cuesta 1, y un falso negativo (un cliente que se predice que va a quedarse en la empresa pero se va) cuesta 10, calcula, en función del umbral utilizado para decidir con el modelo, el coste promedio. Calcula el umbral óptimo de decisión según esa curva.

Semana 3

Paso 1.

En esta semana se usará otro tipo de modelo, el Naïve Bayes, y se realizará una implementación propia. Para ello escribe la clase NaïveBayes, implementando al menos los métodos siguientes:

NaïveBayes
laplace: boolean
.....
fit(X, y)
predict(X): ndarray
score(X, y): score

El atributo Laplace indica si debe o no aplicarse la corrección de Laplace. Este y cualquier otro atributo que necesites debe pasarse al constructor de la clase. Los métodos fit, predict y score implementan una funcionalidad similar a la que hemos comprobado en las librerías de Scikit Learn:

- **fit(X,y):** Calcula el modelo correspondiente a partir de los datos de train. En nuestra implementación se puede pasar un data frame que analice los atributos del dataset y calcule las probabilidades teniendo en cuenta si son valores nominales (tipo object) o numéricos (tipo int, float...). En Naïve Bayes el modelo es el conjunto de verosimilitudes y de probabilidades a priori, calculadas a partir de los datos de train.



- **predict(X):** devuelve el conjunto de clases estimadas para cada uno de los datos de test
- **score(X, y):** devuelve el accuracy medio, es decir, los aciertos en las predicciones del conjunto pasado como argumento.

Paso 2.

Utiliza la clase Naïve Bayes creada en el paso anterior para calcular el score del clasificador en el dataset con el que estamos trabajando en la práctica.

Paso 3.

Scikit Learn ofrece tres aproximaciones del algoritmo Naïve Bayes:

- **MultinomialNB:** es la versión para datos distribuidos multinomialmente. El valor a priori de cada clase se puede asumir uniforme (`fit_prior=False`) o se puede obtener a partir de los datos (`fit_prior=True`). Esta función también permite especificar un parámetro aditivo de suavizado `alpha`. `alpha=0` no realiza ningún tipo de suavizado y `alpha=1` implementa la corrección de Laplace. Este tipo de distribución se utiliza clásicamente en problemas de análisis de texto, como la clasificación de correo en spam/no spam, ya que la distribución multinomial no representa categorías, sino cuentas, por ejemplo, de palabras.
- **GaussianNB:** la verosimilitud de las variables se asume que sigue una distribución normal.
- **CategoricalNB:** es la versión Naive Bayes para datos categóricos. Asume que cada característica tiene su propia distribución categórica. Como **MultinomialNB**, permite indicar los argumentos `fit_prior` y `Alpha`, así como el número mínimo de categorías por característica: `min_categories`.

Clasifica el conjunto de datos con cada uno de los tres algoritmos y compara sus accuracy con el obtenido con la implementación propia. ¿Cuál funciona mejor? ¿Por qué?

ENTREGABLES

- Fichero JupyterNotebook con el código y las respuestas a las preguntas planteadas. En la corrección se tendrá muy en cuenta la argumentación en las respuestas. Se subirá un solo fichero por pareja de prácticas con la nomenclatura **AA1_P3-12_<grupo>_<pareja>.ipynb** (ejemplo: AA1_P3-12_Jueves_1.ipynb)
- Fichero predicciones.csv con los 100 clientes con mayor probabilidad de fuga del fichero **fuga_clientes_empresa_telefonica_explotacion.csv**. El fichero predicciones.csv debería tener solo una columna, con el ID del cliente. Como ejemplo se deja el fichero ejemplo_predicciones.csv